ASSIGNMENT – 9.3

2303A51355

Batch-10

Task-1

Prompt: act as a auto generating function documentation in a shared codebase.

#in a python script with multiple functions without any doc strings. now uh generate google-style function docstrings for each function in the script.

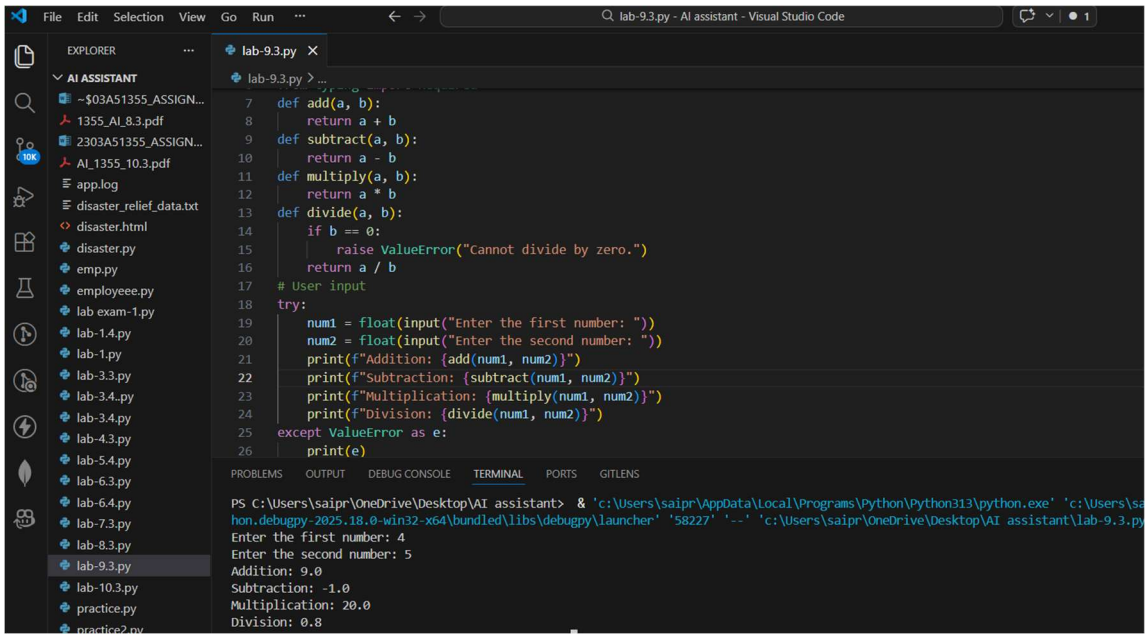
The docstrings should include a brief description of the function's purpose, its parameters, and its return value.

take user input for the functions and print the results.

code :

```
from typing import Required

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

# User input

try:
    num1 = float(input("Enter the first number:

"))
        num2 = float(input("Enter the second

number: "))
```

```
    print(f"Addition: {add(num1, num2)}")
        print(f"Subtraction:    {subtract(num1,
num2)}")
      print(f"Multiplication:   {multiply(num1,
num2)}")
    print(f"Division: {divide(num1, num2)}")
except ValueError as e:
    print(e)
```



Code Analysis:

- The script defines four arithmetic functions: add, subtract, multiply, and divide.
- Each function performs a basic mathematical operation on two numbers.
- The divide() function includes error handling to prevent division by zero.
- The program takes user input for two numbers and calls all functions to display results.
- Google-style docstrings help improve readability and maintainability in shared codebases.

Task-2

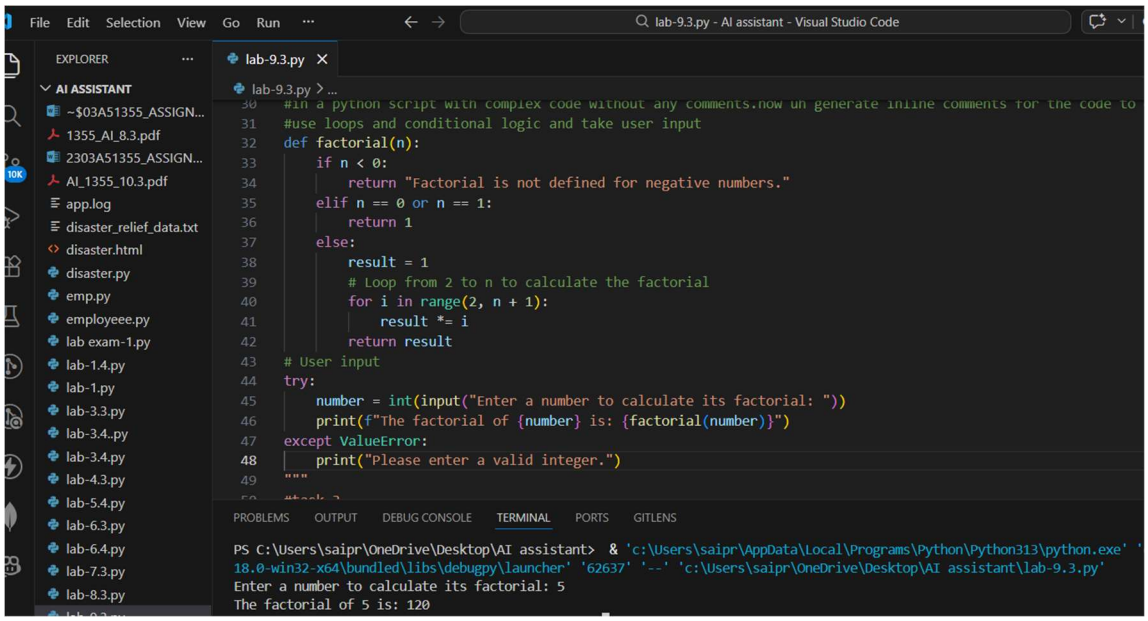
Prompt: act as ai generated inline comments for a complex codebase.

#in a python script with complex code without any comments.now uh generate inline comments for the code to explain the logic and flow of the program.

#use loops and conditional logic and take user input.

Code :

```
def factorial(n):
    if n < 0:
        return "Factorial is not defined for negative numbers."
    elif n == 0 or n == 1:
        return 1
    else:
        result = 1
        # Loop from 2 to n to calculate the factorial
        for i in range(2, n + 1):
            result *= i
        return result
# User input
try:
    number = int(input("Enter a number to calculate its factorial: "))
    print(f"The factorial of {number} is: {factorial(number)}")
except ValueError:
    print("Please enter a valid integer.")
```



Code Analysis:

- The script calculates the factorial of a number using a function.
- It uses conditional statements to handle negative numbers and base cases (0 and 1).
- A loop is used to multiply numbers from 2 to n to compute the factorial.
- The program takes user input and validates it using try–except for invalid integers.
- Inline comments explain the logic and flow of the program step-by-step.

Task-3

Prompt: Write a professional multi-line module-level docstring for the given Python module.

#The docstring should clearly describe the purpose of the module, required dependencies, key classes and functions, and include a short example of how the module can be used.

#Ensure the tone is suitable for an internal repository or real-world project.

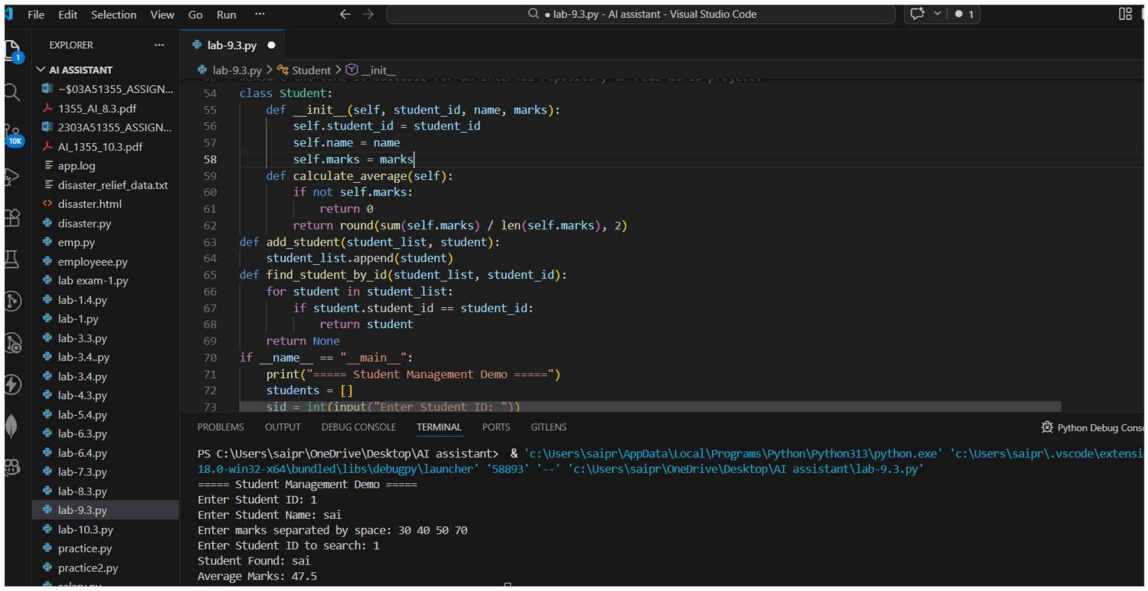
Code :

```
class Student:
    def __init__(self, student_id, name, marks):
        self.student_id = student_id
        self.name = name
        self.marks = marks
```

```
    def calculate_average(self):
        if not self.marks:
            return 0
        return round(sum(self.marks) / len(self.marks), 2)
def add_student(student_list, student):
    student_list.append(student)
def find_student_by_id(student_list, student_id):
    for student in student_list:
        if student.student_id == student_id:
            return student
    return None
if __name__ == "__main__":
    print("===== Student Management Demo =====")
    students = []
    sid = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    marks_input = input("Enter marks separated by space: ")
    marks = list(map(int, marks_input.split()))

    student = Student(sid, name, marks)
    add_student(students, student)

    search_id = int(input("Enter Student ID to search: "))
    result = find_student_by_id(students, search_id)

    if result:
        print("Student Found:", result.name)
        print("Average Marks:", result.calculate_average())
```

else:

print("Student not found")



Code Analysis :

- The module implements a simple student management system.
- It contains a Student class with attributes student_id, name, and marks.
- The method calculate_average() computes the average marks of a student.
- Utility functions add_student() and find_student_by_id() manage student records.
- The program demonstrates usage through user input and searching for a student.

Task-4

Prompt: Convert the existing inline developer comments inside the functions into structured Google-style docstrings.

#Preserve the original meaning of the comments, include description, parameters, return values, and an example where appropriate.

#Remove redundant inline comments after converting them into docstrings to keep the function bodies clean.

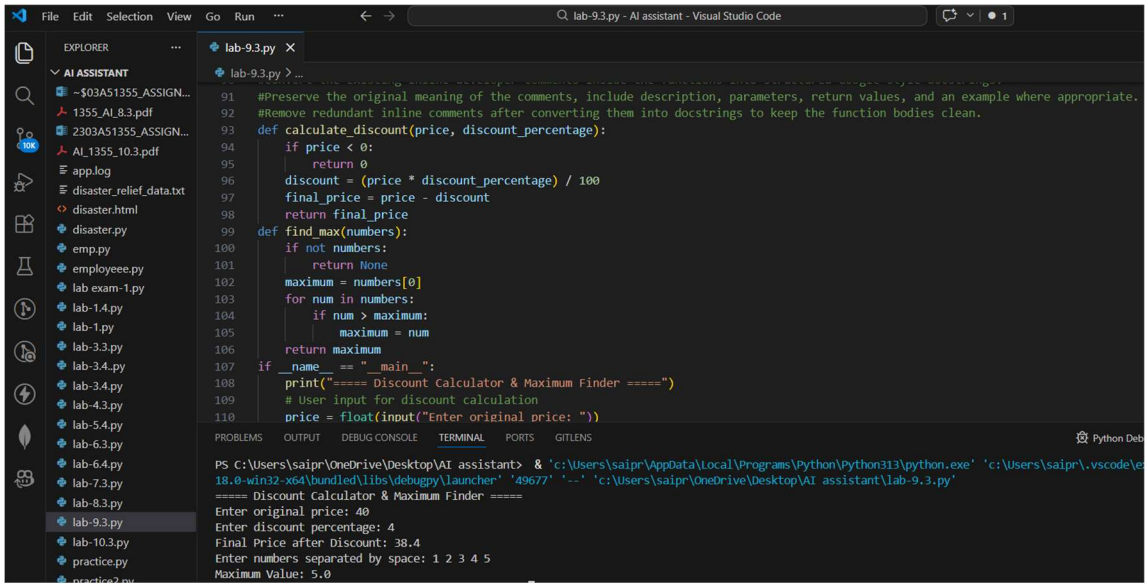
Code :

class ShoppingCart:

```
    def __init__(self):
        self.cart = {}
    def add_item(self, item_name, price):
        if item_name in self.cart:
            self.cart[item_name] += price
        else:
            self.cart[item_name] = price
    def remove_item(self, item_name):
        if item_name in self.cart:
            del self.cart[item_name]
        else:
            print("Item not found in cart.")
    def calculate_total(self):
        return sum(self.cart.values())
# User input
cart = ShoppingCart()
while True:
    action = input("Enter 'add' to add an item, 'remove' to
remove an item, 'total' to calculate total price, or 'exit' to quit:
")
    if action == 'add':
        item_name = input("Enter the name of the item: ")
        try:
            price = float(input("Enter the price of the item: "))
            cart.add_item(item_name, price)
            print(f"Added {item_name} to cart.")
        except ValueError:
            print("Invalid price. Please enter a number.")
```

```
    elif action == 'remove':
        item_name = input("Enter the name of the item to
remove: ")
        cart.remove_item(item_name)
    elif action == 'total':
        total_price = cart.calculate_total()
        print(f"The total price of items in the cart is:
${total_price:.2f}")
    elif action == 'exit':
        print("Exiting the shopping cart program.")
        break
    else:
        print("Invalid action. Please try again.")
```

Output:



Code Analysis :

- The script contains two functions: calculate_discount() and find_max().
- calculate_discount() computes the final price after applying a discount.

 find_max() finds the maximum value from a list of numbers using a loop.

 The program takes user input for price, discount percentage, and numbers list.

 Converting inline comments to Google-style docstrings improves documentation structure and keeps function bodies clean.

Task-5

Prompt: Design a Python utility script that reads a .py file and automatically detects functions and classes.

#Insert placeholder Google-style docstrings for each detected function or class.

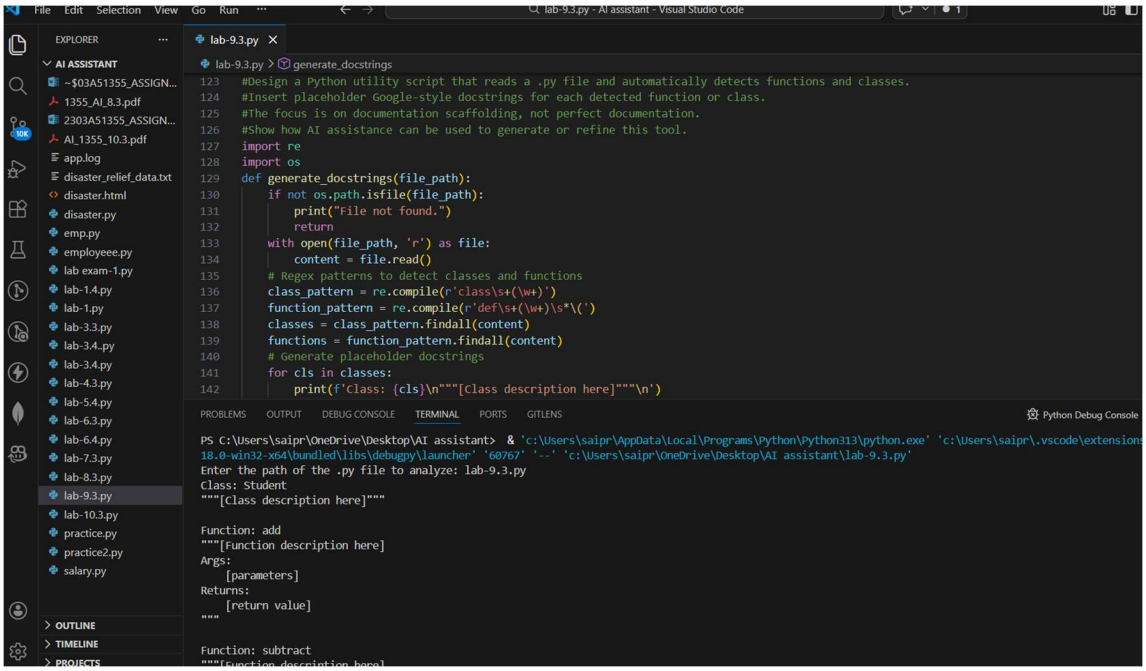
#The focus is on documentation scaffolding, not perfect documentation.

#Show how AI assistance can be used to generate or refine this tool.

```
import re
import os
def generate_docstrings(file_path):
    if not os.path.isfile(file_path):
        print("File not found.")
        return
    with open(file_path, 'r') as file:
        content = file.read()
    # Regex patterns to detect classes and functions
    class_pattern = re.compile(r'class\s+(\w+)')
    function_pattern = re.compile(r'def\s+(\w+)\s*\(')
    classes = class_pattern.findall(content)
    functions = function_pattern.findall(content)
    # Generate placeholder docstrings
    for cls in classes:
        print(f'Class: {cls}\n"""[Class description here]"""\n')
    for func in functions:
                                print(f'Function:       {func}\n"""[Function       description
here]\nArgs:\n    [parameters]\nReturns:\n    [return value]\n"""\n')
```

```
if __name__ == "__main__":
    file_path = input("Enter the path of the .py file to analyze: ")
    generate_docstrings(file_path)
```

Output :



Code Analysis :

- The script reads a Python (.py) file provided by the user.
- It uses regular expressions (regex) to detect classes and functions in the file.
- The program automatically generates placeholder Google-style docstrings.
- It helps developers quickly create documentation scaffolding for large codebases.
- AI assistance can further refine the generated docstrings and improve documentation quality.